

Phase 1 — Data Model

Entity–Relationship Diagrams

The complete Phase 1 database schema for the CPF website rebuild — every entity, field, and relationship, traced back to the live mockup's actual data files, and structured to ship Arabic first with English as a data addition, never a migration.

DOCUMENT

05 — Data Model ERD

SCOPE

Phase 1 only — see §1 exclusions

COMPANION

06 — Data Model UML

Contents

01	Bilingual schema strategy	Translation tables vs. column suffixes
02	How to read these diagrams	Notation & document structure
03	High-level domain overview	Fig. 1 — master entities only
04	Program domain — full detail	Fig. 2 — apps/programs/
05	Content, Home & About domain — full detail	Fig. 3 — activities, news, stories, about
06	Networks & Resources and Forms — full detail	Fig. 4 — makers map, contact intake
07	Traceability: mock data → schema	Every entity's source field
08	What's intentionally absent	Phase 2 exclusions

Bilingual schema strategy

Two realistic ways to store Arabic/English content in Django + Postgres. One was chosen deliberately — verified against a real library's documented schema, not assumed.

APPROACH	HOW IT WORKS	VERDICT
Column suffix e.g. <code>title_ar</code> , <code>title_en</code>	Every translatable field gets one column per language, on the same row.	Rejected as the default. A third language means a real migration across every affected table — <code>Program</code> alone has ~15 translatable fields, doubling or tripling column count. The mockup already does a light version of this (<code>title</code> / <code>titleEn</code>) — workable for a 2-language prototype, not a scaling CMS.
Translation table adopted	Each translatable model gets a matching <code><Model>Translation</code> table: FK back to a language-neutral master row, a <code>language_code</code> field, unique on (<code>language_code</code> , <code>master</code>) .	Adopted. Verified against <code>django-parler</code> 's documented schema — the most widely used Django translation library. Adding a language becomes a data operation, not a migration.

Reference schema (confirmed against django-parler)

```
class Program(models.Model):
    slug = models.SlugField(unique=True)
    # ...language-neutral fields only...

class ProgramTranslation(models.Model):
    master = models.ForeignKey(Program, related_name='translations', on_delete=models.CASCADE)
    language_code = models.CharField(max_length=10) # 'ar' | 'en' | ...
    title = models.CharField(max_length=255)
    # ...other translatable fields...

class Meta:
    unique_together = ('language_code', 'master')
```

WHY LANGUAGE_CODE IS A PLAIN FIELD, NOT A LANGUAGE FOREIGN KEY
A hard FK would force a join on every translation lookup for a value really governed by Django's own `settings.LANGUAGES` . A lightweight `Language` reference table still exists (below) so a Phase 2 CMS can list/manage active languages without a code change — application code validates against it; the translation tables stay a plain indexed string for query performance.

■ PK — primary key □ FK — foreign key ■ UK — unique constraint

How to read these diagrams

The full Phase 1 schema is roughly 40 tables once every translation table is counted — split into one high-level overview and three full-detail domain diagrams, mirroring how the backend itself is split into Django apps (see `03-BACKEND_ARCHITECTURE_AND_SECURITY.md` §2).

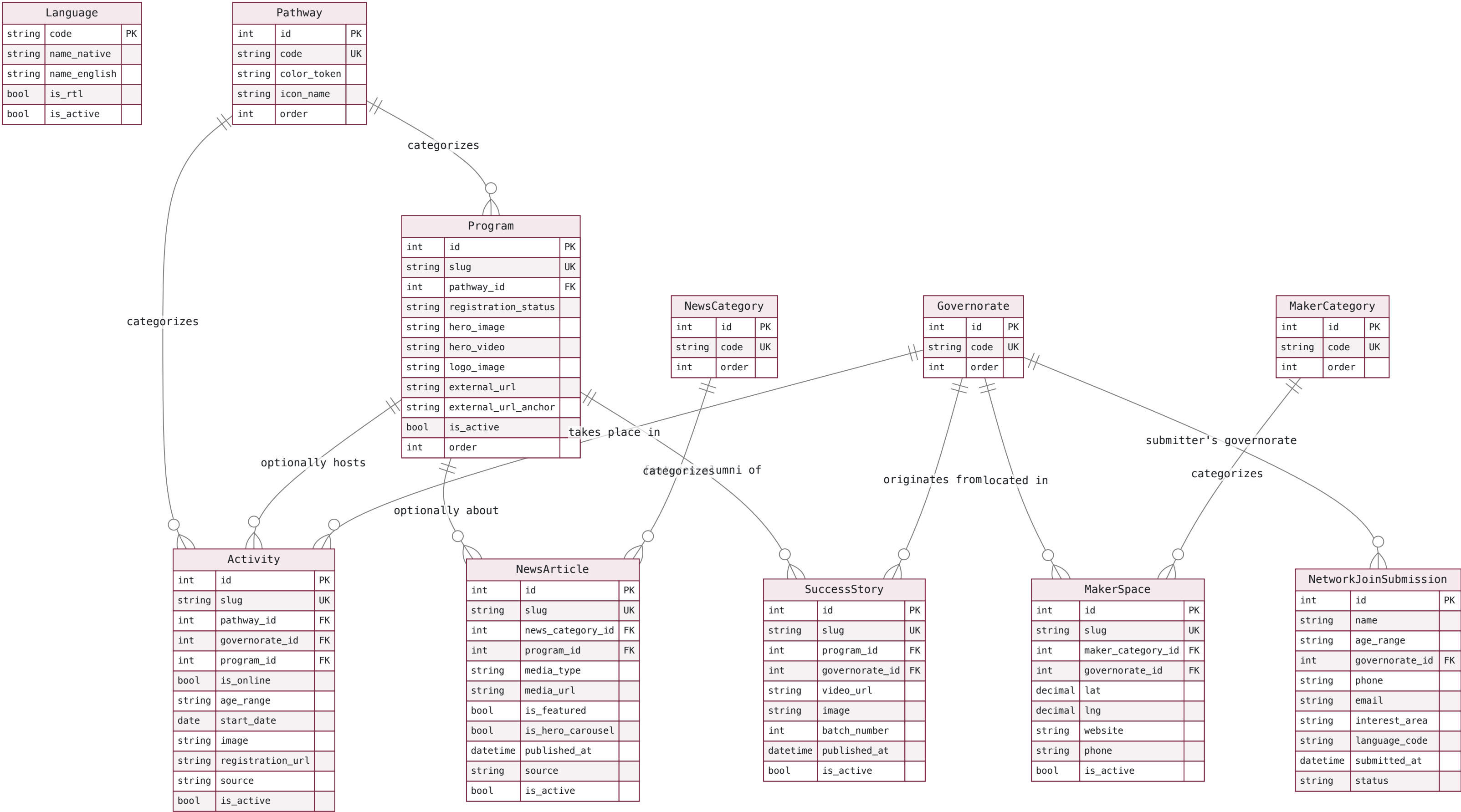
- **Fig. 1 — High-Level Domain Overview:** master entities only (no translation/child tables), for understanding how the domains relate.
- **Fig. 2 — Program Domain:** every table under `apps/programs/`.
- **Fig. 3 — Content, Home & About Domain:** `apps/activities/`, `apps/news/`, `apps/success_stories/`, `apps/about_content/`.
- **Fig. 4 — Networks & Resources / Forms Domain:** `apps/makers_map/` and `apps/contact/`.

Notation: `||--o{'{'}` reads "exactly one, to zero-or-many." `||--o|` reads "exactly one, to zero-or-one" (an optional 1:1).

High-level domain overview

Master entities only — no translation or child tables — to see how the domains actually relate before the detail. **Pathway** (تعلّم / فُند / اصنع الأثر) and **Governorate** (Jordan's 12 governorates) are the two lookups reused most widely; both surfaced only by reading the mock's actual field names rather than guessing a schema.

Fig. 1 — cross-domain relationships (Program, Activity, News, Success Stories, Makers Map, Forms)



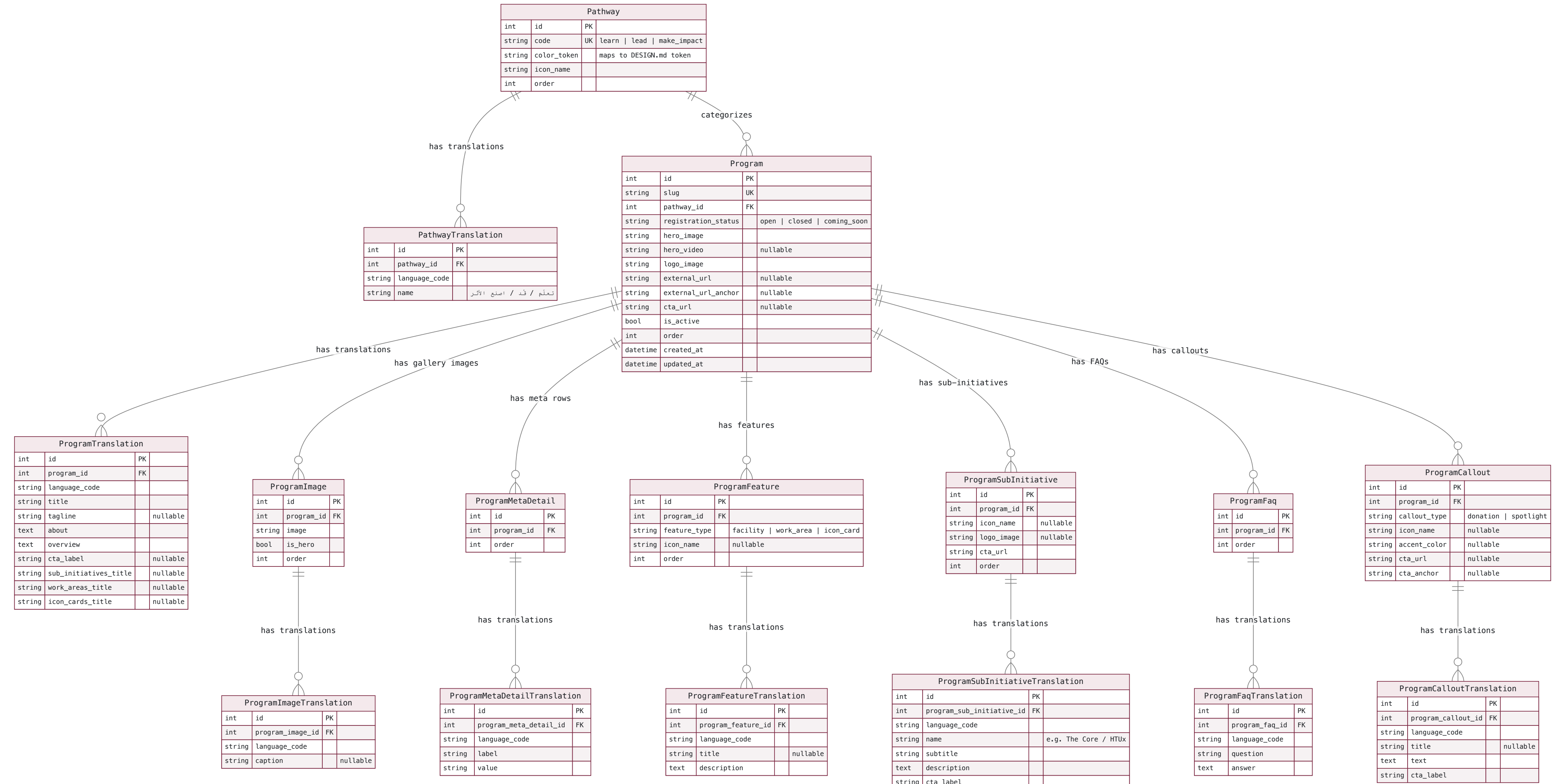
Program domain – full detail

`programsFullData` in the mock is a deeply nested object per program — meta details, facilities, sub-initiatives, FAQs, an optional donation banner, an optional spotlight section, work areas, icon cards. Modeling every nested array as its own bespoke table would produce six near-identical "ordered list of icon + title + text scoped to a program" tables. Two deliberate consolidations avoid that:

- **ProgramFeature** merges the mock's `facilities[]` , `workAreas[]` , and `iconCards[]` . All three are structurally identical — an ordered, program-scoped list of an optional icon + optional title + description — differing only in which page section renders them, captured by a `feature_type` `enum ( facility / work_area / icon_card )`.
- **ProgramCallout** merges the mock's `donationBanner` and `spotlightSection` — both singleton, optional, per-program blocks with an icon/accent, text, and a CTA — via a `callout_type` `enum ( donation / spotlight )`.

`ProgramSubInitiative` and `ProgramFaq` were *not* folded in — a sub-initiative (e.g. "The Core" under HTU) carries its own logo/CTA identity, and an FAQ is a question/answer pair, not an icon+description. Both are semantically distinct enough to earn their own table. `Pathway` lives in code alongside `Program` (`apps/programs/models.py`) — `Activity` in Fig. 3 imports and reuses it rather than redefining it.

Fig. 2 – every table under apps/programs/, including translations · printed at reduced scale to fit the page – see the companion interactive HTML Mermaid source for full-resolution zoom



Content, Home & About domain – full detail

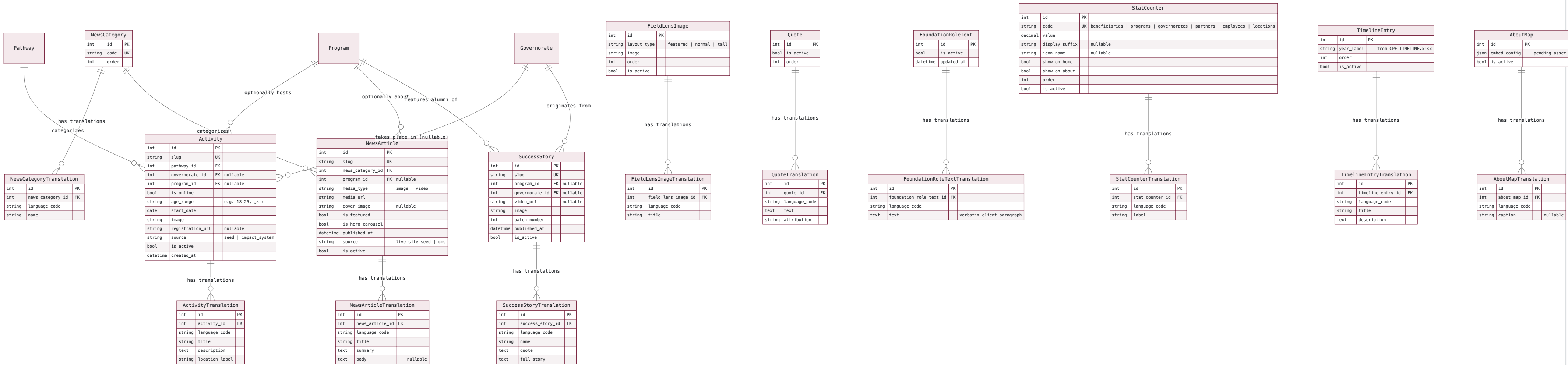
Covers the Home page's Activities section, إبوابة الفرض's Activities tab, News, Success Stories, and every About-page content block. `Activity` and `Program / Pathway` reappear here by reference only — see Fig. 2 for their full definitions.

TWO DECISIONS ONLY VISIBLE BY READING THE MOCK CLOSELY

No `points` column. The mock (`allEvents[]` .`points`) carries a points value on every event. Per the Phase 1 scope document, points/rewards are explicitly excluded this phase — deliberately omitted, not forgotten.

`Activity.governorate_id` is nullable, paired with `is_online`. One mock event uses `city`: ' `أونلاين` ' ("online") instead of a real governorate — rather than force "online" into the `Governorate` lookup as a fake province, it gets its own boolean flag.

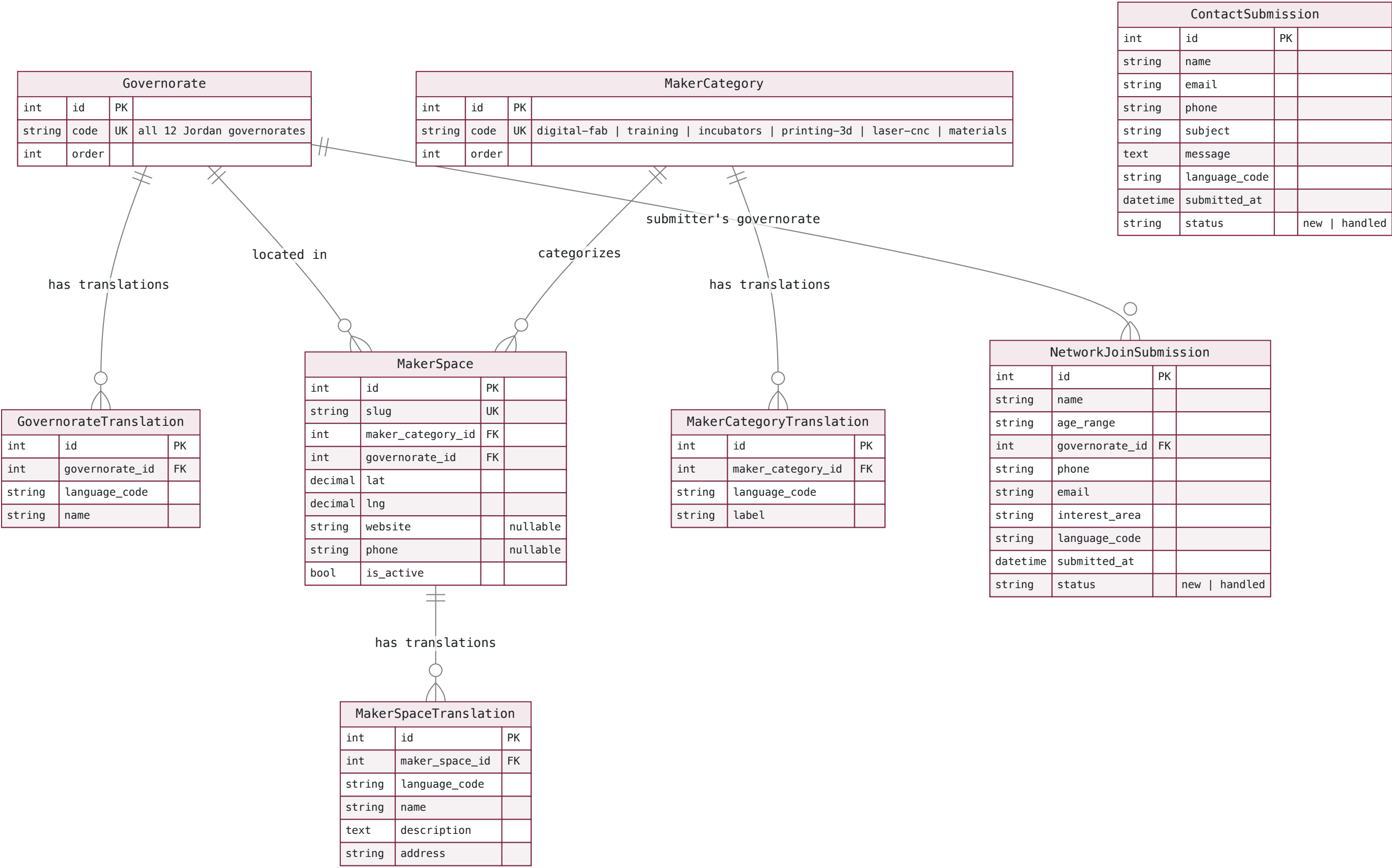
Fig. 3 – every table under `apps/activities/`, `apps/news/`, `apps/success_stories/`, `apps/about_content/` · printed at reduced scale, see note above



Networks & Resources and Forms – full detail

Phase 1's "شبكات وموارد" tab ships **Makers Map only** — Entities Directory and Youth Networks are deferred, so no `Entity` / `EntityCategory` / `YouthNetwork` table appears here or anywhere in this schema.

Fig. 4 – apps/makers_map/ and apps/contact/



`ContactSubmission` and `NetworkJoinSubmission` carry no translation table — both are visitor-submitted data, not CMS-authored content. `language_code` here only records which language the visitor used, for support/reporting.

Traceability: mock data → schema

For anyone auditing whether this schema actually reflects the product — every entity cites the exact mock field(s) it was derived from.

ENTITY	DERIVED FROM (MOCK FILE → FIELD)
Pathway	programsData.js → pathway ('تعلّم' / 'تعلّم' / 'تعلّم' / 'تعلّم'), also on data.js → allEvents[].pathway
Program + ProgramTranslation	programsData.js → programsFullData keys: title / titleEn, tagline, about, overview, image, video, logo, ctaLabel / ctaUrl, registrationStatus
ProgramImage	Not yet in the mock (one image per program today) — added per the Phase 1 scope's explicit "3–4 curated photos per program" requirement
ProgramMetaDetail	programsData.js → metaDetails: [{label, value}]
ProgramFeature	programsData.js → facilities: [], workAreas: [{title, text}], iconCards: [{icon, title, description}]
ProgramSubInitiative	programsData.js → subInitiatives: [{name, subtitle, description, icon, ctaLabel, ctaUrl}]
ProgramFaq	programsData.js → faqs: [{q, a}]
ProgramCallout	programsData.js → donationBanner and spotlightSection
Activity + ActivityTranslation	data.js → allEvents[]: title, date, city, location, pathway, ageRange, image, description (points deliberately dropped)
NewsCategory	newsData.js → newsList[].category / heroSliderNews[].category
NewsArticle + NewsArticleTranslation	newsData.js → newsList[]: title, desc, image, date, isFeatured, programKey; heroSliderNews[]: type, mediaUrl
FieldLensImage	newsData.js → pulseImages[]: type, title, url — "عدسة الميدان"
SuccessStory + SuccessStoryTranslation	allStories[]: name, program / programKey, location, video, image, quote, fullStory
Quote, FoundationRoleText, StatCounter, TimelineEntry, AboutMap	Not literal mock structures — modeled directly from the Phase 1 scope document's explicit About-page content requirements
Governorate	MakersMap.jsx → GOVERNORATES array — all 12 Jordan governorates
MakerCategory	makerSpacesData.js → MAKER_CATEGORIES: [{id, label}]
MakerSpace + MakerSpaceTranslation	makerSpacesData.js → makerSpaces[]: name, category, governorate, description, address, website, lat, lng
ContactSubmission	Contact.jsx general contact form fields
NetworkJoinSubmission	Contact.jsx "انضم لشبكتنا" tab form fields

What's intentionally absent

Consistent with the Phase 1 scope document: no `Partner / Partnership` model, no `Publication` model, no `Entity / EntityCategory` (Directory) model, no `YouthNetwork` model — and, critically, **no points/rewards field anywhere** in this schema, even though the mock's activity data (`allEvents[].points`) has one. If a future ticket asks to "just add it back since we're touching the table anyway" — that's the scope creep this diagram exists to make visible before it happens.